

How to Include Users in the Development of Off-the-Shelf Software: A Case for Complementing Participatory Design with Agile Development

Christina Hansson,
Blekinge Institute of
Technology, Sweden
christina.hansson@bth.se

Yvonne Dittrich
IT-University, Denmark
ydi@itu.dk

Dave Randall
Manchester Metropolitan
University, United King-
dom,
D.Randall@mmu.ac.uk

Abstract

This paper describes and discusses a non-traditional approach to participatory design, one which is combined with an agile-like software development process. In this case, the size of the company combined with a distributed population of users has a serious impact on the software development process. The small software company in our study resolves this problem with an unconventional amalgam of participatory design and agile processes which seems to suit their situation. By using different kinds of user participation the small software provider is able to keep in contact with users on a daily basis. Users convey requirements for new functionalities, give feedback and report errors. Users' feedback and proposals form the basis for further development. The paper relates our observations to other research on participatory design in unconventional settings and discusses the conditions under which agile software development can complement participatory design.

1. Introduction

In this paper, we discuss the relevance of a 'special case' for Participatory Design (PD). The use of PD is one way to involve end-users in the development process. We try to show that PD can be complemented with an agile-like software development process, in certain circumstances, to produce successful user participatory outcomes. The term 'agile-like' is used in order to point out that the process is not a pure agile software process but shows similar characteristics. Our study was designed to evaluate the reasons for the evident success of a small software company selling, supporting and further developing a booking system mainly for the use of municipalities and sports organisations. Idavall Data AB (Idavall), the company in question, is a small company in the south of Sweden consisting of six people.

About 1300 users in roughly 300 customer organisations use the system that forms Idavall's sole product. The company uses a range of methods: support calls, user meetings, courses, the web site and a newsletter to obtain feedback from their users concerning system limitations, faults, and proposals for future development. To be able to make use of the feedback and

change proposals in future development the company developed an agile-like development practice. Our attention was brought to the company fortuitously during a research project around e-government.¹ E-government is about providing public services and information with the aid of new technology, particularly the Internet. This research, when combined with several municipal interests, clearly indicated the importance of the co-development of services, work practices and technology. [5, 6] Ongoing 'design in use', it further suggested, was best served by both flexible and adaptable software and flexible development processes. Often municipalities rely on off-the-shelf software as they cannot afford a custom-made solution. This may be why our ongoing research seems to support the impression that small companies that provide partly custom-made off-the-shelf software often outrun larger software providers. These small companies also often apply flexible development processes and a well-developed relationship to customers and users. Agile software development processes, which are flexible, have a customer-centred approach but lack methods for incorporating real users into the process. By adding PD to the development process it is possible to complement agile software development to secure real users' participation in the process and not only the customer'. Active user participation hopefully guarantees the users' possibilities to influence the system. PD has to be further developed in such contexts and adapted to those circumstances. However, such methodological proposals have to be adapted to fit into the situation and circumstances at hand.

This case study presents adapted and non-traditional PD practices and may be thought of as supporting argument concerning the value of PD but, if so, it is not PD 'by the book'. There are no clear political objectives evident in this collaboration, even if customer organisations could be characterised as having a 'service' ideology. It would thus look much more like socio-technical systems or 'proto' PD than any of the fully fledged, collective resources' oriented approaches. More

¹ *Design of IT in use - supportive technologies for public services* (DiTA), funded by the Swedish Agency for Innovation Systems VINNOVA, April 2000 - December 2002 (project no. 2001-03659)..

importantly, the participation process itself cannot be characterised in terms of any of the orthodox treatments of ‘user involvement’. There is something ‘ad hoc’ about the various methods, they suggest and ‘extreme’ responsiveness insofar as different channels of communication are used as circumstances require and various responses quickly generated. Our article is a contribution to the field of participatory design in unconventional settings [9, 12, 22] which questions whether some aspects of PD are appropriate in such contexts. Thus, the practices we observed can help us to expand the toolbox of PD. The observations reported here not only provide an interesting contrast between ideal-typical and academic, approaches to PD, and ‘real world’, situationally driven, approaches, they also concern the relation between PD and software development. We provide a more nuanced view of what the relationship between PD and software development might be. We need to find methods where we can bridge the gap between PD and the more technical side of the software process. Combining an agile- or agile-like software development process with PD might be a fruitful method. An agile-like development process, in this case, does not refer to any special agile method; it refers more to a non-traditional way of developing software inspired by the family of agile methods [4]. Especially small companies could learn from this way of developing software and services.

We begin by briefly describing our fieldwork methods, outlining the situation at Idavall, the evolved methods they have developed for obtaining user feedback and the software development practice that evolved to accommodate user feedback and change proposals. We then discuss whether that practice can be regarded as participatory design and briefly describe agile software development. In the discussion, we identify methods and practices that might be transferable to other contexts and relate them to relevant research results in the discourses of participatory and agile software development.

2. Fieldwork Methods

The research we describe here was mainly based on a pragmatic approach which we might loosely term ‘evaluative ethnography’ and on interview and observation

The fieldwork at Idavall began in January 2002. We visited Idavall regularly during the spring. Specific methods used included semi-structured and open-ended interviews, participatory observations at the company, field notes and analysis of documents. We also participated in a couple of user meetings and training courses. Video recordings were used to check and enrich our observations. In the fall 2003 we returned to Idavall in order to follow up on our previous fieldwork.

2.1 The qualitative study

Qualitative research as we see it focuses on understanding how people work things out rather than on

measuring their performance in accordance with abstract criteria. In other words, it does the work that Winch [23] demanded of social theory, i.e. it produces results which can be generalised in terms of the ‘rules’ that people use to get things done as opposed to the ‘causes’ which supposedly determine their behaviour. The analysis and evaluation of the field material yields, in this sense, understandings which can be generalised about how people organize their practice. Rich descriptions – sometimes misunderstood as unscientific storytelling – should allow the readers to follow and, if necessary, argue the conclusions [20].

By using multiple ways of collecting data and combining different kinds of methods it is possible to support one finding with the help of others. We validated our impressions from the fieldwork by constantly referring back to, and cross-checking against various informants.

The focus of the study was to understand how Idavall is able to accommodate the frequent changing requirements of its users and customers and how it was that distributed users were able to participate in the development process. We were also interested in to what extent the users felt they could influence the development process. This focus is mirrored by the structure of the present article as described above.

3. User participation in Idavall

Idavall was founded in 1987. In the early years, the company developed a number of different programs, but from 1991 they focused primarily on the booking system referred to here as FRI. Customers are widely dispersed, mainly in Sweden, but also in Finland and Norway.

FRI is one of the most frequently used booking systems in Sweden. The most important customers are the Swedish municipalities, where a large number of different municipal administrations use the booking system. The software has a web interface that complements the basic FRI. The booking system is responsible for the administration of invoices, admission control and subsidies as well as for bookings.

Idavall’s avowed objective is to keep contact with the users, and to let their feedback guide the future development. The user in this case means the one who really uses the program, as distinguished from ‘customers’. Both the terms ‘customer’ and ‘user’ in this article are used in the same way with the contrast between those who pay and those who do the work in mind.

FRI was designed at the outset for one specific user. Since then the user community has expanded dramatically. This expansion has not altered the fundamental business concept employed at Idavall, which is to listen to users and develop the software in a way that continues to keep customers satisfied. Gary, one of the employees expresses his standpoint at a demonstration of FRI as follows: “The development is driven by our users not by ourselves”. In pursuance of this, representatives from Idavall meet their users through different kinds of activities. Following is a description of these different activities.

ties. This description forms the basis of coming discussions.

3.1 User meetings

Every year about 8 to 10 meetings for users and customers are held throughout Sweden, Norway and Finland. The meetings are informal, and their purpose is to disseminate news, discuss further development, and answer questions about the booking system. For the users, user meetings provide the opportunity to meet other users in the same area. User-meetings thus offer an opportunity to create networks that make it easier to get in touch with each other and thus co-operate around common questions or problems. Those who participate in user meetings think they are a good opportunity to learn new functions and meet other users and representatives from Idavall. Those who do not participate think that they do not need to or that they do not have the time. Some prefer to have private teaching and support. One said "I do not want to listen to other users' problems; and who is interested in listening to our problems?"

Idavall's representatives encourage users to present proposals for new functionality and to report errors. The following translation from the field is an example of such a user proposal.

Example 1

A user wants to lengthen the text field of the bank account number in FRI because the last figures in the number disappear if the number is too long. He says that he has to write the number by hand on the invoice otherwise. He suggests that it should be possible to adjust this. Gary promises that this proposal will be implemented in the next release of FRI. Later he says that he knows that it ought to be an advantage to other customers as well and it will be easy to implement.

The above example shows a specific user suggestion and the promise associated with it. Further conversation with Gary elicited the view that, while it is important to respond to user needs, it is equally important to be honest. He is particularly concerned not to make promises that cannot be kept and as a consequence, he seldom commits himself without first discussing with the developers.

3.2 Support

One of the most important parts of Idavall's business philosophy is to offer a adequate, friendly and professional support. Users think that the support service provided is very good, they always get quick and personal help. Explanations of problems are easy to understand. In addition to the user meetings, user support offers one of the most important methods of keeping informed

about users' needs, wishes and proposals. Idavall claims that the objective is to talk to the user in exactly the same way that users normally talk, and avoiding technical jargon. No one, as one respondent said, 'should feel stupid or crazy when calling Idavall for support'. Support is given Monday to Friday between 8 am and 12 am. Everybody answers support calls, even the developers of the team. This in turn means that the developers receive first-hand feedback about problems with their product thereby removing a reporting problem.

A call to the support service is initially generated when a problem arises. During the course of such calls, proposals for new functionalities often appear. The difficulty of the users' question can vary from a simple question about how to change a booking to more difficult technical questions. Almost every phone call to the support service is logged in a text database. When someone calls in, the call taker at Idavall searches for the caller's name in order to connect to the support history of this specific person. The call taker can look up similar problems and solutions in the database. During, or directly after, each phone call the call taker inserts notes about the call.

3.3 Courses

FRI can easily be tailor-made to specific requirements. How it is used depends on the customer; a higher level of knowledge is required than for using Microsoft Windows and Office™. Idavall offers courses where the use of different parts of the booking system is discussed and taught. Users state that they consider it important to participate in courses. Besides learning about the booking system, they suggest they also get to know other users, which in turn makes it easier to share knowledge, instance e.g. by calling and asking how other organizations adapt the program to a specific task. They also like to come to Idavall and meet the developers. 'It becomes easier to call the support service when you know the face of a person'. Below is an example from a Web-association course.

Example 2

The leader of the course, Jason, who is one of the developers at Idavall, talks about functionalities and shows how to do different set-ups to make the application fit the users requirements. During the course the participants came up with proposals, one wanted, for example, to have a text field to be able to fill in the phone number of the office in the register of the associations. Jason told the user that this had already been implemented and showed it to the user. Another wanted the text on the information page to be formulated differently because the present formulation was a bit unclear. Jason discussed this proposal together with the other participants and agreed on a new formulation. Jason encourages users

to call him if they run into problems after the course.

4. The software development

The development of FRI is an ongoing process. The system is continually being improved to satisfy the ever-evolving needs of users. How do the developers at Idavall manage to make use of and accommodate all the error reports and change proposals from their users? Despite the absence of a formal development process, the process can be seen as two different cycles. The faster and smaller on-going development cycle where errors are corrected and minor improvements continually take place is highly flexible. In the larger and slower long-term development cycle, major improvements take place. These cycles run simultaneously throughout the year.

Planned formal meetings where the development of the booking system is discussed are rare. Informal and spontaneous meetings are, on the other hand, very common. As everyone is located in the same place and only a few people are involved, it is easy to meet and talk when a problem or question arises. Coffee- and lunch breaks serve as important opportunities for common discussions. Spontaneous meetings in front of the computer are used to solve implementation problems and discuss improvements.

4.1 Deciding what to do

Idavall, as we have seen, has a valuable ongoing dialogue with users via the support service, user-meetings, courses and similar activities. As mentioned earlier, it is through these activities that users present their change proposals. Before the implementation of new functionality starts, the proposals are reviewed and informally ranked by the staff. Proposals are ranked according to their quality: Is the change generic? How would it affect other functionality? Is it useful for many users? And how cumbersome will it be to implement the change? Ted, one of the developers, said that he preferred to implement many smaller improvements rather than one large one because many smaller changes make a lot of people happy. As every developer also has contact with users and customers, their perspective is shared by many.

4.2 Daily ongoing development cycle

The focus here is on implementation of users' proposals, refinements of existing functionalities, improvements in existing parts and correcting errors. Correcting errors has the highest priority. Programming takes place primarily in the afternoons when the support service is closed. It takes between one and five weeks for a new version to be released on the website. A 'What's new' description is published for each version which describes what kinds of changes have been implemented. This allows users to

choose for themselves whether or not to download the most recent version. The system is designed so that not every change needs to be downloaded and installed. When a change is small only the program modules need to be updated; the database remains unchanged. The idea is that every user shall be able to download and install a new version without the help of a technician. However new versions are in most cases installed by a technician; this is often because FRI is hosted on a server..

Once a year or when necessary, a major release is delivered. This is announced on the website and every user receives an e-mail. This updates the database and program modules. All workstations must also be updated.

4.3 Long-term development cycle

The last major development took place in 1996, when a module was added that introduced steering number code locks throughout the system. Today a new 32-bit version of FRI, which will replace the present 16-bit version, is being developed by Ted. The development of this new version normally takes place when the smaller development cycles are relatively quiet. The new version will offer opportunities to add several new features, e.g. the automatic sending of an e-mail to confirm a booking. Today one must copy a confirmation and paste it into an external e-mail program before it can be sent. It is impossible to implement some of the change proposals in the present software due to an outdated implementation technique. The new version will accommodate the new improvements. Some users think that the 16-bit version is a little old-fashioned and has some technical disadvantages.

A beta-version is already available for users interested in testing it. These users give feedback; this influences the ongoing development. The old version of the software will be maintained and run parallel with the new version for an indefinite time. It is not clear when the new version will be released but there is a large demand for it. The length of the delay depends on how long it will take to produce a reliable version. As the maintenance and improvement of today's version have priority, planning is difficult.

5. How to understand Idavall's practice

The method described above of interacting with users when it comes to the further development of the software and development process can be understood, we claim, as a combination of non-traditional PD and non-traditional software engineering. By non-traditional PD, we mean a form of PD that not take place in the form of organized workshops as part of a nicely cut development project. The software development process, we observed, is also different to the traditional way of developing software. However, this non-traditional method of involving users in the development process suits the situation at hand, and the software development process is necessary to accommodate the users' feedback and change proposals.

We use our experiences of Idavall's method of developing software and discuss how it is possible to combine PD and a more agile way of developing software to develop and maintain a fruitful co-operation between developers and users.

First, we discuss PD practices. Then we briefly describe agile software development and its background, and relate it to the development practices we observed. In the section that follows, we discuss the central measures for taking in feedback and design proposals from a dispersed user community and for combining PD with an agile-like software development process; we also discuss to what these measures can be generalized.

5.1 An introduction to participatory design

PD is considered to be a Scandinavian approach to system design that is strongly tied to thoughts and principles regarding democracy within the workplace. 'This Scandinavian approach might be called a work-oriented design approach. Democratic participation and skill enhancement – not only productivity and product quality – themselves considered ends for the design'. [7, p. 41]

A traditional PD approach has two important characteristics. The first is of a political nature, and deals with questions of workplace democracy and control over the workplace. PD has an explicit agenda for organizational and political change [14]. The second is of a technical nature, and states that the participation of skilful users in a development process contributes to successful design and high quality products. PD is a learning process where the user and the system developer, or the designer, learn from one another [7].

Traditionally, PD has a number of guiding principles, and a range of techniques that are applied in practice. The principles include the acknowledgement of diversity, ensuring the real impact of the participants' viewpoints in the design process, mutual learning at all levels, separation of interest groups, and protection of participants. Methods include visiting work places, creating work scenarios, future workshops and organizational games, mockups and prototyping [22].

Tools and methods used in PD are to a large extent based on early projects concerned with developing methods for 'participative analysis of relations between work and technology and strategies for union influence over technology projects run by managers and IT professionals' [7, p. 174]. These tools and methods have a common goal: to provide users and designers with methods for connecting current and future work practices with visions of a future technology, by allowing access to the experiences of both users and designers [7]. An objective for participating in the process of PD is that it gives the designer new and better ways of gaining an understanding of the user's everyday work practices. [21] Design professionals need knowledge of the actual use context and employees need knowledge of possible technological options. These types of knowledge are developed most

effectively through active cooperation between employees and designers. [14]

5.2 A non-traditional way of conducting participatory design

That there is something like user participation going on in the development of the booking system cannot be disputed, but it is carried out in a way which is non-traditional way for PD. Users affect the development through user-meetings, technical support and courses, but they never actually participate in design activities like workshops or mock-up evaluation. Users feel that they have an opportunity to affect developments. Very few users express sentiments concerning Idavall's responsiveness. What is going to be implemented is ultimately determined by the developers at Idavall. User feedback is the key factor in the development process. The importance of user feedback is also recognised in PD literature. Developing software systems without listening to the users may be possible, but it is unlikely that such a system will live up to users' requirements. Feedback is also needed during the entire life cycle of a system. It ensures that a system is continually improved and adapted to changing work practices. [14]

Traditional PD methods do not fit into the situation described above since users are located at different work places all over Scandinavia and because the small size of the company precludes large-scale, dispersed user involvement. A large and dispersed population of users, and the co-operative use of mock-ups and prototypes is likely to be outdated before they have even been produced. Moreover, small work places, companies or organisations do not always have the opportunity to carry out projects where users and designers can work together over a long period to develop a system uniquely suited to the tasks, practices and environments of its users. Instead, this company has evolved a tailored form of user participation which allows it to function and users' needs still to be brought into the design process. [22] Such activities as user meetings, courses and the support service took over this role in the development process described. The present system takes the place of activities like prototyping and builds a base for further discussions and development. During discussions users come up with proposals based on the needs in the day-to-day work situation. These proposals are used as input for forthcoming versions and part of the evolutionary development. The support service and courses work in a similar way.

Toerpel, et al. [22] describe a similar case of a networked organisation where traditional PD methods are not applicable. They recommend instead a continuous process of parallel experimentation and network-wide collection of experience, feedback, and integration into an overarching infrastructure consisting of a variety of local substructures when participants are not able to meet for bigger design sessions, or when the infrastructure does not allow that. They believe that an ongoing

process of contributions and discussion in smaller groups will have a positive effect on the on-going development of a system. The smaller groups they describe can be compared to what Fischer [8] called *Communities of Interests* (CoI). Another concept introduced by Fischer and Ostwald [9] is *informed participation* which can be regarded as an extension of PD. Informed participants work in different work places, but have same problems that are comparable to others in similar or different workplaces. A basic challenge facing the CoIs is the development of a shared understanding of the task at hand. New knowledge is constructed through discussions and mutual learning. Participation shifts in such circumstances from designing a system to using and evolving it. User meetings and courses are activities that usually bring users together and help to develop CoIs.

The measures taken by Idavall allow the users of the booking system to influence the development of the software and as such their individual working situation. We suggest that the traditional methods and tools for PD must be adapted to suit the employees and customers needs and practices in order to allow for user participation even, as in our case, in the development of off-the-shelf software.

However, our case also shows that the development process must complement user participation. There is an obvious need for flexible development processes that allow quick response to error reports and change proposals. Below we discuss similarities between agile software development - a highly flexible methodology that implements a customer-centred approach - and the observed development practice.

5.3 An introduction to agile software development

Agile software development can be seen as a reaction against too rigorous and bureaucratic development processes. In the early years of software development many organizations and companies developed software without any recall to a process model. This worked well as long as systems were small and not too complex. As programs grew in size and complexity, different methods were developed to make software development more predictable and efficient. Strong emphasis was put on the planning process. One of the best known methods is the waterfall model that was first described in 1970 by Winston Royce. [18] The waterfall model emphasizes development through sequential phases with predefined documents and milestones. The customer is only involved before the project starts and when it comes to testing the final product. The end-user is not mentioned at all. [18].

Iterative process models have been developed to cope with the short comings of that model, especially when requirements of the software and its interaction with the environment cannot be defined in advance. (See [1] for an example.) To allow for a co-development of work practices and technology, Floyd et al developed an evolutionary model for software development. [10] However,

even today many practitioners and researchers applying and developing PD and use-oriented development accept that user participation should be limited to the 'early phases'. [13]

Agile software development has developed as a practitioner's answer to the software development models oriented towards control that are proposed by the mainstream of software development. [4] Agile software development is a methodology that aims to make software development more flexible and focuses on highly flexible environments with quickly changing requirements. [4] An emphasis is put on flexible planning that allows adaptation to changes in business and technology. Agile software development implements a highly customer-centred approach where the customer plays a central role during the design process. Projects involve customer feedback on a regular basis. The proponents of this method recommend close customer-developer co-operation and communication in a highly iterative, prototype-based process that provides early feedback on development efforts. The aim is to deliver working software on a regular basis, every couple of weeks or at least every couple of months. The working software is the primary measure of progress. [18]

Discussions centred on agile development open prepare the ground for reconsideration of the relationship between PD and software development. Rittenbruch et al. [17] used agile software development combined with participation of a dispersed user community. The early versions invited for highly qualified feedback to an innovative application.

In the light of the discussion centering on agile development, the practices we observed at Idavall can be described as agile development practice: Software is developed in small cycles. Idavall publishes about 20 revisions per year via its website. User feedback is collected via the support line and user meetings. The developers themselves take part in these activities. The change proposals are prioritised in accordance with their value they provide for users. Major re-factoring efforts take place alongside and through the smaller development cycles. Idavall's example also shows that user participation cannot in always be organised independently of the development process: Without a quick reaction to error reports and improvement proposals, users would not have the confidence that their feedback is taken seriously.

The development practices at Idavall, as well as the project reported about in [17] also highlight that changes in the technical development process are not enough. If user involvement is taken seriously, the organization of user participation when combined with an agile process requires additional measures beyond those is proposed in the related literature. Developers, on the other hand, who use agile software development processes often lack a more thorough knowledge of the context and the users where the system will be implemented. In the literature cited above, the customer is represented by a person who may or may not have a sound understanding of the use context of the software. An obvious risk is that one repre-

sentative cannot sufficiently convey the users' needs and requirements to the development team.

Idavall has developed a practice which involves collecting user feedback and change proposals to inform an agile-like software development practice that allows developers to react quickly and in a flexible manner. In the following section we discuss the tools and means used by Idavall to involve diverse and dispersed users in the agile development of the booking system and discuss their applicability in a wider context.

6. Making use of feedback and design proposals from a dispersed user community

What can we learn from Idavall's practice of combining agile development with participatory design? We identify four tools or methods. Firstly, Idavall's support service is designed to be informal, friendly and encouraging. Even developers take part in providing the service. Users do not only report problems and receive help, they also chat about possible improvements. Secondly, a database is used to keep track of problems and develop a common resource for remedies; it is also used to keep track of change proposals. Thirdly, with the aid of courses, user meetings and a newsletter, the user community provides Idavall with feedback and design input from experienced users. It is from this community that beta version testers are recruited. Fourthly, with an agile-like development process the developers can accommodate the users' feedback and change proposals in a flexible way.

6.1 Using support as a channel for feedback and change proposals

Idavall uses the support service as an input for error reports, general feedback and change proposals. According to Idavall's policy everybody, even the developers are involved in the support service. It is considered important that the developers know their users and to receive feedback and change proposals from those who actually use the software. The developers also have the opportunity to ask for clarifications and additional information. Errors can be corrected immediately, and the source of irritation is limited. The close relationship between developer and user allows the latter to feel comfortable in suggesting new features and complaining if necessary. Often, support calls generate proposals for new functionality. There might, for instance, be a wish for an extra text field to be added to be able to fill in a mobile phone number. Another proposal can be to increase the opportunity to change different settings in the web application. Almost all proposals, errors and change requests are taken to the coffee table and discussed.

The support service is widely used to generate error reports which will be handled by the maintenance department. Pool et al. [16] report on the deployment of support service knowledge to provide more comprehen-

sive input in the form of user stories for an agile maintenance process. Having said this, the support service is seldom if ever used to generate input for further design and development. It might be possible to use this support service more systematically as a communication channel for involving users in the development process.

6.2 Using Customer Relationship Management tools to keep track of user feedback

To keep track of user feedback and change proposals, particularly where many people use the software, some kind of computer-based system is helpful. Idavall uses a database to store information about customers, users, and support calls. As every phone call is documented, the call taker can look up the version the user has installed and earlier problems reported from the same place. The log is also used to collect change proposals and other feedback when planning implementation work.

Idavall's database can be seen as a customer relationship management tool. Customer Relationship Management (CRM) is a concept used in the economic marketing area. The aim of CRM is to create a strong and lasting relation between supplier and customer. [2] Computerized systems that manage customer relationship are commercially available. Marketing and sales departments are the main users of these systems.

CRM tools may be used as complements to PD, especially when feedback from a great number of customers and users has to be taken care of. Idavall shows that the use of CRM methods and tools must be backed up by a commitment to react to error reports, feedback and change proposals.

6.3 Developing a user community

Idavall organises courses and user meetings, and distributes a newsletter. Users are encouraged to help each other and share knowledge about special ways of using the software. The mutual exchange and the way feedback is taken into account during future development allow a user community to grow. People are engaged in discussing possible future developments, and they allow the testing of early versions of the software when a long-term development cycle comes to a close. Users share the practice of using FRI, and they have a common interest in improving the software so that it adjusts to their developing needs. They can be regarded as a community of interest, even if it is a dispersed one.

Similar user communities can be observed in the computer game industry. [11, 12] Expert users receive preview versions of new developments and become involved in designing new features. As the computer gamers feel at ease with electronic communication, the establishment of a website virtual community seems sufficient.²

² Virtual communities are digital intermediaries which can support continuous interaction between different social interest groups on the Internet or, for example, between software providers and users. [3]

In both cases, the commitment of the development organisation to take design proposals from the community into account when developing the software provides an important incentive for users to become involved.

6.4 Using an agile-like software development process

The processes and practices of Idavall closely resemble those practices usually spoken of as agile software development. In small delivery cycles, developers focus on corrections and improvements; whereas redevelopment takes place within a longer time frame. These development practices allow developers to take into consideration the feedback of the users. Users steer the development even when they are not on site; requirements for new functionality are derived from the users' feedback; prioritization of requirements takes place on the same basis; new code is tested and integrated on a daily base. Developers share a common coffee and lunch area and communication is very close. Informal discussions about design are a daily event. The deviation can be motivated through the specific situation at Idavall. The program is used by a widely dispersed and somewhat diverse group of customers and users. The development team is extremely small.

Participatory design has been connected to evolutionary software processes that allowed for learning cycles which enable users to experience software in the early stages and give qualified feedback for further design. [10] Other researchers experienced and discussed the suitability of agile development and especially extreme programming when combined with participatory design. [17] However, as Rittenbruch et al. also discussed, agile software development methods have to be adapted to fit participatory design. The Idavall case exemplifies how it is possible to use agile software development to complement PD. Here more empirical research and experience are needed.

7. Possible risks of this way of development

As proposed above, Idavall's practices combine successfully user participation with an agile-like software development process. Users are highly involved in the development process, and the developers can react quickly to new requirements. But the observant reader might wonder whether everything is really so unproblematic. And of course, one can find problems; and the development process has disadvantages and involves risks as well. These can have a serious impact on the development process and cause problems. We see the following potential problems.

The prioritization process is totally controlled by the developers at Idavall, users are only able to propose new functionality but have no impact on the process. In the end, only certain parts of the user community participate in the development process and thereby have an impact on the system. These users may not be typical users and

are thus not representative. This is a potential problem since the wrong proposals may be implemented. This is the opposite of what happens in open source projects. In these projects new proposals are tested heavily by users before a new version is released. [15]

The informality of the development practice and the fact that only six people are employed at Idavall makes the development process highly dependent on the individual person, and thus vulnerable. Every employee has his or her own specialties and cannot be easily replaced. It takes a long time to learn how the different parts of a booking system are built up and how to support them. According to Gary, it takes about one year before you are able to give FRI support in a proper way and on your own. If someone falls ill for a long period there would be serious problems. Idavall is aware of this but is not able to hire more employees at the moment. Program developers at Idavall take care of support, develop the program, teach courses and participate in user meetings. To implement such a practice one must find employees who are able to rule out both the technical and social aspects of the work. It is difficult to find the right people for the job. Gary told us that the last time he had to find someone to employ who would fit into the company it took him more than one year.

One last question is whether Idavall's development process scales up regarding the size of the software, the size of the development team and the size of the user community. In any case, it provides an example of how use-oriented software development can take place, and as such provides a basis for the adaptation of PD, agile software development and the measures discussed in the previous section to other situations.

8. Conclusions

Small companies in particular could learn from this way of using agile development processes and PD as a means of becoming more sensitive to customer and user requirements. They also could learn how to systematically gather user feedback and make use of it in the development process.

- The members of a dispersed user community have a major impact on a software development process. Users steer development in the sense that they give feedback on an existing program and make proposals for new functionality. This is made possible by the different arrangements where users and developers regularly interact with each other, face-to face and by telephone. These arrangements serve not only to provide information to the company, but also to generate a sense of importance and value among users and customers in such a way that they generate a 'community of interest' by talking to each other and exchanging useful ideas.
- The database – the CRM tool - also plays an important role in the development process by providing a starting point for the kinds of interaction mentioned above.
- Traditional PD still focuses on the working place and the people who will use the new system, i.e. the users.

The more technical side of the development process is not taken into consideration sufficiently. It is important to understand the technical process to be able to fully accommodate PD and the technical side of software development.

- Developers who use agile software development processes, on the other hand, often lack a more thorough knowledge of the users and the use context of the system. The customer is represented by a person who may or may not have a sound understanding of the users and the use context of the software.
- A combination of PD and agile-like software development, where both user context and development issues are taken into consideration could stimulate the creation of a more comprehensive picture of how a user-oriented software engineering process for off-the-shelf software could look.
- An agile-like software development process makes development highly flexible and sensitive to the environment in which the software is used. User feedback can be responded to at short notice. Flexible software and processes are of special interest in the area of e-government where a lot of different end users, e.g. citizens, administrators and local developers have diverse requirements and needs. The software is brought into line with customers' and users' needs.

However, more empirical research and experience are needed to further understand and develop agile development processes complementing PD.

9. Acknowledgments

Thanks are due to the users of FRI and members of staff at Idavall for their kind co-operation. A special thanks our colleagues for inspiring discussions. Also thanks to the E-participation project financed by the foundation of the savings bank Kronan and run by prof. Sara Eriksén, Blekinge Institute of Technology.

10. References

- [1] Boehm, B. W. A Spiral Model of Software Development and Enhancement, *IEEE Computer*, May, 1988
- [2] Brown, S A. (ed.) *Customer Relationship Management: A Strategic Imperative in the World of E-Business*, Wiley, UK, 2000
- [3] Chang, A-M., Kannan, P.K., Whinston, A. B. Electronic Communities as Intermediaries: The Issues and Economics. In: *Proceedings of the 32nd Hawaii International Conference on System Science*, IEEE Computer Society Press, Los Alamitos, CA, 1999
- [4] Cockburn, A. *Agile software development*, Addison-Wesley, Boston, 2002
- [5] Dittrich, Y., Eriksén, S., Hansson, C. PD in the Wild: Evolving Practices of Design in Use. In: Binder, T., Gregory J., Wagner I. (eds). *Proceedings of the Participatory Design Conference*, CPSR, Malmö, Sweden, 2002
- [6] Dittrich, Y., Ekelin, A., Elovaara, P., Eriksén, S., Hansson, C. Making e-Government happen: Everyday co-development of services, citizenship and technology. In: *Proceedings of the 36th Hawaii International Conference on System Science*, IEEE Computer Society Press, Los Alamitos, CA, 2003
- [7] Ehn, P. Scandinavian Design: On Participation and Skill. In: Schuler, D., Namioka, A. (eds.): *Participatory Design: Principles and Practices*, Hillsdale, New Jersey, 1993, pp. 41ff.
- [8] Fischer, G. Communities of Interests: Learning through the Interaction of Multiple Knowledge Systems. *24th Annual Information System Research Seminar in Scandinavia (IRIS'24)*, Ulvik, Norway, 2001
- [9] Fischer, G., Ostwald, J. Seeding, Evolutionary Growth, and Reseeding: Enriching Participatory Design with Informed Participation. In: Binder T., Gregory J., Wagner I. (eds). *Proceedings of the Participatory Design Conference*, CPSR, USA, 2002
- [10] Floyd, Ch., Reisen, F. M., Schmidt, G. STEPS to Software Development with Users. In: Ghezzi G, Mc Dermid J. A. (eds). *ESEC '89* Berlin, 1989
- [11] Henfridsson, O., Holmström, H. Developing E-commerce in internetworked organizations: A case of customer involvement throughout the computer gaming value chain. *Database for Advances in Information Systems*, vol: 33, issue: 4, 2002, pp. 38-50
- [12] Holmström, H. Virtual Communities as Platforms for Product Development: An Interpretive Case Study of Customer Involvement in Online Game Development. In: *Proceedings of ICIS 2001, 22nd International Conference on Information Systems*, New Orleans, LA, USA, 2001
- [13] Kensing, F. Simonsen, J., Bødker, K. MUST - A method for Participatory Design. *Human-Computer Interaction*, Vol. 13, no. 2, 1998, pp. 167-198
- [14] Kensing, F., Blomberg, J. Participatory Design: Issues and Concerns. In: *Computer Supported Cooperative Work 7*, Kluwer Academic Publishers, Netherlands, 1998, pp. 167-185.
- [15] Locke, J. *Open Source Solution for Small Business Problems*, Charles River Media, Hingham, Massachusetts, US, 2004
- [16] Pool, C. Huisman, J W. Using Extreme Programming in a maintenance Environment, *IEEE Software*, Vol. 18, no. 6, 2001, pp. 19-26,
- [17] Rittenbruch, M., McEvan, G., Ward, N., Mansfiels T., Bartenstein, D. Extreme Participation – Moving Extreme Programming Towards Participatory Design. In: Binder, T., Gregory, J., Wagner, I. (eds). *Proceedings of the Participatory Design Conference*, CPSR, Malmö, Sweden, 2002
- [18] Robert C. Martin Agile Development: Principles, Patterns, and Process. In Robert C. Martin *Agile Processes*, Prentice Hall, 2002.

- [19] Royce, W. *Software Project Management, A Unified Framework*, Addison-Wesley, USA, 1998
- [20] Rönkkö, K., Lindeberg, O and Dittrich, Y., 'Bad Practice' or 'Bad Methods'. Are Software Engineering and Ethnographic Discourses Incompatible? In: *2002 International Symposium on Empirical Software Engineering (ISESE 2002)*.
- [21] Shuler, D., Namioka, A. (eds.). *Participatory Design: Principles and Practices*, Lawrence Erlbaum Associates, Hillsdale, New Jersey, 1993.
- [22] Toerpel, B., Wulf, V., Kahler, H. Participatory Organizational and Technological Innovation in Fragmented Work Environments. In: Dittrich, Y., Floyd, C., Klichewski, R. (eds). *Social Thinking – Software Practice*, MIT Press, USA, 2002
- [23] Winch, P. *The Idea of a Social Science*, Routledge and Kegan Paul, London, 1958